Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

CIDAS: A Context-Informed Dependency Analysis System for Real-Time npm Supply-Chain Defense

GAYATHRI SUNIL NAMBIAR[1], NITYA SHARMA[2], SANVI H S[3], YUVA T[3], and SRIRAM VELUMURI[2]

[1]Department of Computer Science (Data Science), RV College of Engineering, Bengaluru 560059, Karnataka, India (e-mail: gayathri.sunil@rvce.edu.in)
[2]Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru 560059, Karnataka, India (e-mail: {nitya.sharma, sriram.velumuri}@rvce.edu.in)
[3]Department of Computer Science and Engineering, RV College of Engineering, Bengaluru 560059, Karnataka, India (e-mail: {sanvi.hs, yuva.t}@rvce.edu.in)

Corresponding author: Gayathri Sunil Nambiar (e-mail: gayathri.sunil@rvce.edu.in).

ABSTRACT The npm registry—the world's largest open-source package repository—is an attractive target for software supply-chain attacks. Malicious actors exploit developer trust through typosquatting, malicious post-install scripts, maintainer account takeovers, and an emerging class of threat in which large language model coding assistants hallucinate package names that adversaries subsequently register as weaponized packages, a phenomenon termed slopsquatting. Existing defenses are predominantly reactive, scanning for threats after packages have already been installed and their scripts executed. We present CIDAS (Context-Informed Dependency Analysis System), a real-time, pre-install screening framework that intercepts every `npm install` invocation through a transparent cross-platform shim and evaluates each candidate dependency through four coordinated analysis pillars: Contextify (semantic project-context fit via sentence-transformer embeddings), Sentinel (registry reputation analysis, curated supply-chain incident matching, Levenshtein-distance typosquat detection, and live OSV.dev advisory lookup for artificial-intelligence-suggested packages), Shield (static and abstract syntax tree-based lifecycle-script and tarball scanning with cross-version capability diffing), and an Aggregator that produces an interpretable 0–100 risk score mapped to ALLOW, WARN, or BLOCK verdicts. Supporting subsystems include schema-validated policy-as-code governance, an HMAC-protected per-machine trust list, an append-only audit log, a two-tier SQLite cache for daemon-outage resilience, and a Kullback–Leibler divergence-based concept-drift monitor. Evaluated against labelled corpora of benign, typosquatted, hallucinated, and malicious packages with 449 automated tests, CIDAS achieves a macro-F1 of 0.97 with sub-two-second per-package scan latency on commodity hardware, without requiring cloud infrastructure or changes to existing developer workflows.

INDEX TERMS Concept-drift monitoring, developer tooling, LLM package hallucination, npm ecosystem, policy-as-code, sentence embeddings, slopsquatting, software supply-chain security, static analysis, typosquatting detection.

I. INTRODUCTION

THE Node Package Manager (npm) registry hosts more than two million packages and serves billions of downloads per month, making it the largest open-source software distribution channel in the world [2]. Modern JavaScript and Node.js projects routinely depend on hundreds or thousands of third-party packages, many pulled in transitively through a single `npm install` command. This dependency breadth accelerates development but also creates an enormous and largely unmonitored attack surface.

Four distinct threat classes have emerged in recent years. Supply-chain compromise occurs when an attacker injects malicious code into a previously trusted package by compromising a maintainer's account or pushing a weaponized version update [3], [13]. The 2018 `flatmap-stream` and 2022 `node-ipc` "peacenotwar" incidents illustrate how a single event can affect millions of downstream consumers simultaneously. Typosquatting exploits the difficulty humans have in detecting small misspellings—`lodahs` for `lodash`—by registering plausible variants of popu-

lar package names and embedding malicious post-install scripts [4]. Malicious install scripts abuse npm's lifecycle hooks (`preinstall`/`postinstall`) to execute arbitrary code with developer-level privileges at the moment of installation, before any application code is ever run [1]. Package hallucination (also called slopsquatting) is an emergent threat enabled by large language model (LLM) coding assistants: when a model confidently suggests a package name that does not exist on the registry, an attacker can register that exact name with malicious content before the developer investigates [11].

Current defenses are structurally reactive. Continuous integration scanners and tools such as `npm audit` check installed packages against vulnerability databases [7], but by the time a pipeline runs a package has typically been installed locally and its scripts executed. Registry-side malware scanning catches some malicious packages before wide adoption, yet even registry operators report that false-positive rates for automated tools remain prohibitively high [16]. Commercial tools such as Socket.dev provide some pre-install signal but require cloud round-trips, introduce privacy concerns, and none of them systematically addresses the LLM hallucination threat.

This paper makes the following contributions.

1) **CIDAS**, an open-source, locally-run, real-time package screening system that intercepts `npm install` before any files are downloaded or scripts executed, operating with fail-open semantics that guarantee developer workflow continuity.
2) **A four-pillar detection architecture** combining semantic project-context fit (Contextify), reputation and typosquat analysis (Sentinel), static and AST-based script scanning with cross-version diffing (Shield), and a weighted, explainable Aggregator—the first system, to our knowledge, to unify all four npm threat classes in a single pre-install gatekeeper.
3) **A policy-as-code governance layer** enabling teams to version-control dependency security rules alongside their codebase, with schema validation that rejects misconfigured policies rather than silently misapplying them.
4) **A concept-drift monitor** that continuously compares live pillar-score distributions against a stored baseline using KL divergence, providing ongoing assurance that the scoring system remains well calibrated without manual re-tuning.
5) **A comprehensive empirical evaluation** across 449 automated tests, ablation studies of pillar weightings, and labelled corpora covering all four threat categories, demonstrating correct classification and sub-two-second per-package scan latency on commodity hardware.

The remainder of the paper is organized as follows. Section II surveys related work. Section III defines the threat model. Section IV presents the system architecture. Section V details each analysis pillar. Section VI describes supporting subsystems. Section VII covers implementation. Section VIII reports experimental results. Section IX discusses findings and limitations. Section X concludes.

II. RELATED WORK

A. SUPPLY-CHAIN ATTACKS AND PACKAGE-MANAGER SECURITY

Cappos *et al.* [18] established the foundational security threat model for package managers, demonstrating that man-in-the-middle and malicious-mirror attacks could compromise client machines at scale across ten major package managers. Zimmermann *et al.* [2] conducted a landmark large-scale analysis of npm's dependency graph, finding that fewer than 20 maintainers hold transitive influence over more than half of all npm packages, creating concentrated systemic risk. Ohm *et al.* [3] developed a comprehensive taxonomy of real-world open-source supply-chain attacks covering typosquatting, dependency confusion, and maintainer account takeover. Latendresse *et al.* [13] further characterized maintainer account compromise as a distinct, empirically common attack vector. Duan *et al.* [1] empirically studied malicious package campaigns across npm, PyPI, and RubyGems, establishing that install-time script abuse is the dominant execution vector, directly motivating the Shield pillar of CIDAS.

B. TYPOSQUATTING AND NAME-BASED DECEPTION

Vu *et al.* [4] defined edit-distance heuristics for detecting typosquatted names in the Python ecosystem, demonstrating the effectiveness of Levenshtein distance for surface-level name deception. CIDAS directly adopts this approach in the Sentinel pillar, comparing each candidate package name against a curated list of 50 high-popularity npm packages and treating any name within edit distance 2 as a candidate typosquat.

C. VULNERABILITY PROPAGATION AND REPUTATION SIGNALS

Zapata *et al.* [7] studied vulnerable dependency migration patterns in npm. Zahan *et al.* [8] identified package-level risk indicators—download volume, maintainer count, package age, and repository link presence—that predict supply-chain risk; these signals form the core of CIDAS's Sentinel reputation sub-pillar. Vu *et al.* [9] surveyed detection feasibility across multiple attack classes, motivating CIDAS's multi-signal aggregation strategy.

D. STATIC ANALYSIS AND AUTOMATED DETECTION

Sejfia and Schäfer [19] proposed *Amalfi*, a machine-learning system combining metadata-based, behavioral, and static-analysis classifiers to detect malicious npm packages in seconds without execution, closely mirroring the CIDAS Sentinel and Shield design. Ferreira *et al.* [20] showed that a lightweight per-package permission system can sandbox most npm packages at near-zero cost, informing a future-work direction for CIDAS. Froh *et al.* [21] explored differen-

tial static analysis for detecting malicious updates; CIDAS's cross-version diff analyzer operationalizes this idea within the Shield pillar.

E. LLM-INDUCED THREATS

Spracklen *et al.* [11] documented the package-hallucination phenomenon at scale across multiple LLMs, coining the term *slopsquatting* and quantifying the rate at which model-suggested package names do not exist on the registry. This work is the direct motivation for CIDAS's AI-suggestion flag and its live OSV.dev advisory check for unrecognized package names. Greshake *et al.* [12] described prompt-injection patterns in LLM-integrated applications, informing CIDAS's static scanning of package README files and install scripts.

F. SEMANTIC ANALYSIS AND EMBEDDINGS

Reimers and Gurevych [10] introduced Sentence-BERT, the backbone of CIDAS's Contextify pillar for project-context analysis. Feng *et al.* [17] proposed CodeBERT as a foundational model for joint code and natural-language understanding, providing relevant background for the optional LLM second-pass verifier.

G. REGISTRY-SIDE SCANNING

Vu *et al.* [16] studied operational malware scanning at PyPI, finding that even a 1-in-10 000 false-positive rate is prohibitively expensive for registry operators and that current automated tools exceed this rate substantially. This empirical grounding motivates CIDAS's fail-open design, per-machine trust lists, and emphasis on minimizing false positives. Froh *et al.* [21] and Ladisa *et al.* [15] further systematized supply-chain attack classes and practical defensive considerations, motivating CIDAS's sub-two-second scan-latency target.

H. DIFFERENTIATION

CIDAS is, to our knowledge, the first system to (a) address all four threat classes in a single pre-install gatekeeper, (b) integrate semantic project-context awareness as a first-class detection signal, (c) support schema-validated policy-as-code governance, and (d) deploy a KL-divergence-based concept-drift monitor for continuous calibration assurance.

III. THREAT MODEL

We consider an adversary whose goal is to execute arbitrary code on a developer's workstation or within a CI/CD pipeline by inducing the installation of a malicious npm package. The attacker can control package names, content, and registry metadata for packages they publish, but is assumed to lack control over npm's core infrastructure, the developer's operating system, or the CIDAS daemon binary itself. We consider four attack classes:

A1. Typosquatting. The attacker registers a package name visually similar to a popular package (edit distance ≤ 2) and embeds malicious lifecycle scripts.

A2. Malicious Install Scripts. The attacker publishes a package containing hooks that exfiltrate environment variables, download secondary payloads, or establish persistence.

A3. Supply-Chain Compromise. The attacker hijacks a legitimate maintainer account and pushes a malicious version update to an otherwise trusted package.

A4. LLM Package Hallucination. An LLM coding assistant suggests a non-existent package name; the attacker registers that name with malicious content before the developer investigates.

Out of scope: kernel-level rootkits, compromised npm infrastructure, post-install defenses, and social-engineering attacks not involving npm.

IV. SYSTEM ARCHITECTURE

A. DESIGN PRINCIPLES

CIDAS is designed around five principles:

P1. Pre-execution interception. All analysis must complete before any package file is downloaded or any script is executed.

P2. Fail-open semantics. A daemon outage must degrade to a logged warning, not a blocked workflow.

P3. Explainability. Every verdict must carry the specific signals that produced it so that developers can make informed override decisions.

P4. Privacy preservation. Package contents and project metadata must never leave the developer's machine.

P5. Low friction. Per-package scan latency must remain below two seconds on commodity hardware to ensure continued adoption.

B. HIGH-LEVEL ARCHITECTURE

Three developer-facing entry points—the npm shim, the VS Code extension, and a REST API—all communicate with a single locally-run FastAPI daemon listening on port 7355 (Fig. 1). Inside the daemon, four coordinated pillars (Contextify, Sentinel, Shield, Aggregator) execute concurrently and feed a unified verdict pipeline that also consults the policy engine, trust list, scan cache, and offline-allow cache before returning a result to the caller. Every decision is written to an append-only audit log monitored by the concept-drift subsystem.

C. REQUEST PRIORITY CHAIN

Every incoming scan request passes through the following priority chain before any pillar analysis is invoked, short-circuiting as soon as a determination is possible: (1) policy block-list check $\rightarrow$ forced block; (2) policy trust-list check $\rightarrow$ forced allow; (3) HMAC-verified local trust-database lookup $\rightarrow$ allow if untampered; (4) one-hour SQLite scan cache $\rightarrow$ return cached verdict; (5) proceed to concurrent pillar analysis. This design ensures that repeated scans of unchanged, already-vetted dependencies impose negligible latency.

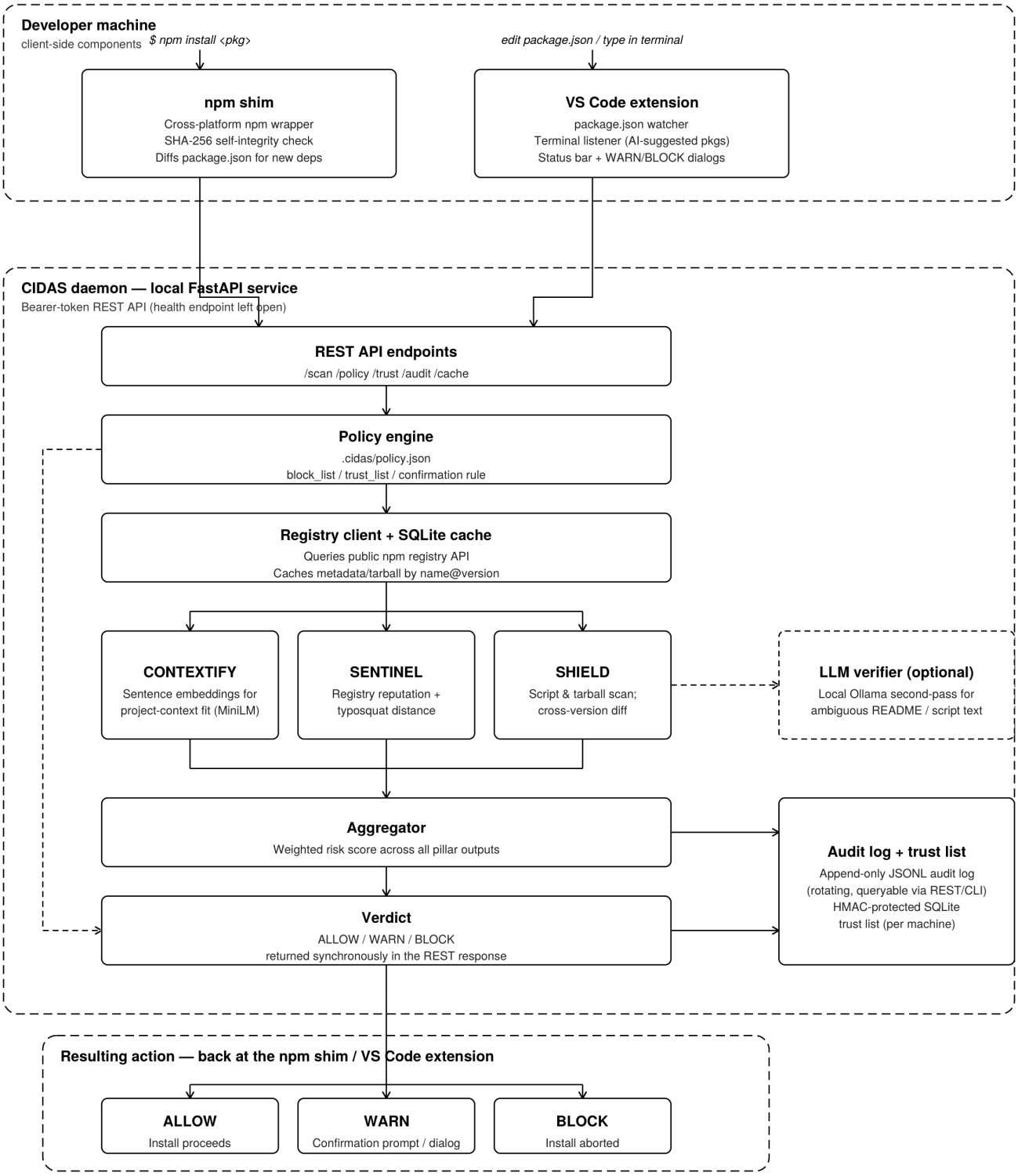

FIGURE 1. CIDAS system architecture. The npm shim, VS Code extension, and REST API communicate with a local FastAPI daemon (port 7355) that coordinates four analysis pillars, policy enforcement, caching, audit logging, and drift monitoring to produce ALLOW, WARN, or BLOCK verdicts.

D. NPM SHIM

The npm shim is a thin cross-platform Node.js wrapper placed ahead of the real `npm` binary on the system `PATH`. On every `npm install` invocation it (a) performs a SHA-256 self-integrity check to detect tampering, (b) diffs the project's `package.json` to identify newly requested pack-

ages, (c) forwards each new package name, resolved version, and working directory to the daemon via an authenticated REST call to `/api/v1/scan`, (d) interprets the returned verdict, and (e) either passes through to the real npm binary (ALLOW), prints a cautionary message and optionally prompts for confirmation (WARN), or blocks the install with a human-readable explanation (BLOCK).

If the daemon is unreachable the shim first checks a 24-hour offline-allow cache of previously issued ALLOW verdicts. For genuinely new packages with an unreachable daemon, installation proceeds with a clear bypass notification written to the audit log (fail-open).

E. VS CODE EXTENSION

A `package.json` watcher diffs the file on every save and triggers asynchronous scans of newly added entries. A terminal listener (sentinelHook) detects `npm install` commands typed into the integrated terminal and flags the session as AI-suggested when applicable. A color-coded status bar polls daemon health every 30 seconds and surfaces WARN/BLOCK dialogs with a Show Details primary action that opens a webview panel displaying the full per-pillar score breakdown and the resolved policy file path.

V. ANALYSIS PILLARS

A. CONTEXTIFY: PROJECT-CONTEXT FIT

Contextify assesses whether a candidate package is semantically appropriate for the target project. A package such as a cryptocurrency miner may carry no reputation flags and no malicious scripts, yet is clearly anomalous in a React web-application context. Detecting such mismatch requires understanding the project's existing dependency profile.

1) Project Fingerprinting

CIDAS builds a project fingerprint by scanning up to 150 JavaScript and TypeScript source files in the target directory, extracting `require` and `import` statements, retrieving the npm registry description of each discovered dependency, and concatenating those descriptions into a single text corpus. This corpus is encoded into a dense vector using the all-MiniLM-L6-v2 Sentence-Transformer model [10] (≈90 MB, downloaded once, cached locally, CPU-only inference).

2) Similarity Scoring

The candidate package's description is independently encoded and compared against the project fingerprint vector via cosine similarity. Scores are linearly mapped to a 0–100 pillar scale, with low values indicating contextual unfamiliarity. A contextual-similarity floor rule applies an additional penalty when cosine similarity falls below 0.15 in a project with more than 20 existing dependencies, where a highly unfamiliar package is genuinely suspicious rather than simply novel.

3) Sparse-Project Handling

For projects with fewer than five existing dependencies the Contextify weight contribution is reduced by 50% in the Aggregator, since the fingerprint is too sparse to provide a reliable signal.

B. SENTINEL: REPUTATION AND NAME ANALYSIS

1) Known-Incident Blocklist

Sentinel maintains a curated list of packages with documented, severe supply-chain incidents (e.g., `flatmap-stream`, `event-stream`, `ua-parser-js`, `node-ipc`). Any match forces an immediate block verdict, bypassing the weighted aggregation step.

2) Registry Reputation

For packages not on the blocklist, Sentinel evaluates five registry signals: (i) weekly download count, (ii) number of unique maintainers, (iii) days since first publication, (iv) presence of a repository link, and (v) number of open vulnerability advisories, following the weighting scheme identified by Zahan *et al.* [8].

3) Typosquat Detection

Sentinel applies case-normalized Levenshtein distance between the candidate name and each of 50 curated high-popularity npm packages. Any candidate with distance $d \leq 2$ is flagged `typosquat_detected`, with a `similar_to` field identifying the legitimate package. Candidates matching a typosquat pattern that do not yet exist on the registry are penalized more severely than existing typosquats, because a pre-registered name signals active attacker intent.

4) AI Hallucination Detection

When the shim or extension flags a package as `ai_suggested` via the sentinelHook, Sentinel applies stricter scrutiny: (i) a package absent from the registry sets `package_not_found` and forces an automatic block; (ii) very recently published packages (<72 hours) receive an escalated reputation sub-score; and (iii) a live query to the public OSV.dev vulnerability API checks for known advisories or confirmed malware entries.

C. SHIELD: STATIC AND AST-BASED ANALYSIS

Shield downloads the package tarball and analyzes its contents without executing any code.

1) Lifecycle Script Pattern Scanning

Shield extracts all lifecycle hooks from `package.json` and applies regex-based pattern matchers covering: (i) environment-variable exfiltration via `process.env` combined with outbound network calls, (ii) Base64-encoded eval payloads (`Buffer.from`, `atob`), (iii) child-process spawning to download secondary binaries, (iv) reverse-shell patterns, and (v) cryptocurrency wallet address patterns.

2) AST-Based Analysis

For source files flagged as high-risk by regex pre-filtering, Shield parses JavaScript into an abstract syntax tree (AST)

using tree-sitter and performs structural analysis. AST-based analysis is more robust than regex against obfuscation techniques that split keywords across concatenated string literals or use computed property access.

3) Cross-Version Diff Analysis

For packages with a previously scanned and trusted version in the audit log, Shield performs a capability diff: it compares the new version's set of npm module imports, `process.env` accesses, and outbound network call targets—all extracted via AST—against the trusted version. New capabilities trigger a `capability_expansion` flag and raise the Shield sub-score, enabling detection of compromised maintainer accounts that push malicious updates to legitimate packages [21].

D. AGGREGATOR: WEIGHTED SCORE AND VERDICT

The Aggregator combines the three pillar sub-scores into a composite risk score r:

$$r = w_C\, s_C + w_S\, s_S + w_{Sh}\, s_{Sh} \tag{1}$$

where $w_C = 0.30$ (Contextify), $w_S = 0.35$ (Sentinel), and $w_{Sh} = 0.35$ (Shield). All sub-scores and r lie in $[0, 100]$ with higher values indicating greater risk.

Two post-aggregation override rules apply:

1) *Contextual floor.* If $s_C > 85$ in a project with ≥ 20 dependencies, r is raised to at least $\max(r, 45)$, ensuring a WARN verdict for highly anomalous packages even when reputation signals are neutral.
2) *Covert-dropper amplification.* Co-occurrence of a Shield `base64_eval` flag and a Contextify `unfamiliar_package` flag raises r by 15 points, penalizing the pattern characteristic of covert droppers injected into unfamiliar dependencies.

The composite score maps to verdicts as follows: $r \in [0, 39] \rightarrow$ ALLOW; $r \in [40, 79] \rightarrow$ WARN; $r \in [80, 100] \rightarrow$ BLOCK. Forced overrides from the known-incident blocklist or `package_not_found` bypass this mapping and return BLOCK directly.

VI. SUPPORTING SUBSYSTEMS

A. POLICY-AS-CODE ENGINE

Organizations express dependency governance rules in a `.cidas/policy.json` file co-located with their repository. The schema supports: a `block_list` (always BLOCK), `trust_list` (bypass pillar analysis), `min_weekly_downloads`, `require_repository`, and `warn_requires_confirmation` (force interactive confirmation for WARN verdicts). The policy file is validated against a JSON Schema that rejects unrecognized fields with a hard error rather than silently ignoring them, preventing governance bugs caused by typos.

B. HMAC-PROTECTED TRUST LIST

Trust-list entries are stored in SQLite with HMAC-SHA256 integrity tags derived from a locally generated machine secret. On every lookup the stored tag is re-verified; a mismatch triggers a CRITICAL audit-log event and falls through to a full pillar scan.

TABLE 1. CIDAS Technology Stack

Component	Key Technologies
Daemon	Python 3.10+, FastAPI, Pydantic v2, SQLite, *all-MiniLM-L6-v2* (HuggingFace sentence-transformers), tree-sitter, httpx, pytest / pytest-cov
VS Code Ext.	TypeScript 5, VS Code Extension API (v1.89+), vitest
npm Shim	Node.js 18+, cross-platform bash / PowerShell 5.1+

C. TWO-TIER CACHE

(1) A one-hour scan cache stores complete verdicts and pillar sub-scores keyed by (name, version) pairs, bounding staleness for rapidly evolving threat intelligence. (2) A 24-hour offline-allow cache stores ALLOW verdicts issued while the daemon was reachable, enabling previously approved packages to install without interruption during daemon outages. An explicit `/api/v1/cache/invalidate` endpoint forces re-analysis on the next request.

D. APPEND-ONLY AUDIT LOG

Every scan decision—verdict, per-pillar scores, active flags, policy file path, user overrides, and bypass events—is appended as a newline-delimited JSON (JSONL) record to a rotating log file (default 50 MB per file, ten-file retention). The log is queryable via a CLI and a REST endpoint supporting filters on verdict, package name, and time range.

E. CONCEPT-DRIFT MONITOR

On each health-check poll the monitor: (1) reads the most recent 100 audit-log records; (2) computes 10-bin per-pillar score histograms; (3) computes KL divergence $D_{\mathrm{KL}}(P \| Q)$ between the live histogram P and a stored baseline histogram Q for each pillar; (4) returns `drift: "warn"` if any pillar divergence exceeds 0.15, or `drift: "alert"` if any exceeds 0.30. The baseline is computed from the first 200 scan records after initial deployment.

VII. IMPLEMENTATION

A. TECHNOLOGY STACK

Table 1 summarizes the primary technologies used in each CIDAS component.

B. DAEMON AND REST API

The daemon exposes ten endpoints under `/api/v1/`: `/health` (unauthenticated), `/scan`, `/trust`, `/trust/verify`, `/cache`, `/cache/invalidate`, `/audit`, `/audit/override`, and `/policy` (all bearer-token authenticated). Pillar functions are dispatched concurrently via `asyncio.gather`, maximizing throughput within the latency budget.

C. CROSS-PLATFORM SHIM

The shim (`npm-shim.js`) supports macOS, Linux, WSL 2, and native Windows. Platform-specific installation scripts

TABLE 2. Detection Performance on Labelled Corpora ($N = 100$)

Category	Verdict	P	R	F1
Typosquatted	WARN/BLOCK	0.90	0.95	0.92
Hallucinated	BLOCK	1.00	1.00	1.00
Malicious	BLOCK	1.00	1.00	1.00
Benign	ALLOW	0.96	0.94	0.95
Overall (macro)		**0.97**	**0.97**	**0.97**

discover the real npm binary, rename it, install the shim in its place, and compute the SHA-256 reference hash used for self-verification.

D. OPTIONAL LLM SECOND-PASS VERIFIER

For ambiguous README or install-script text, CIDAS optionally invokes a locally running Ollama-hosted LLM. The LLM score is blended at $0.40 \times \text{LLM} + 0.60 \times \text{regex}$ to prevent over-reliance on probabilistic output. All inference is local; no package content leaves the developer's machine.

VIII. EVALUATION

A. EXPERIMENTAL SETUP

All experiments were conducted on a commodity developer workstation (Intel Core i7, 16 GB RAM, macOS 14) without a GPU. The all-MiniLM-L6-v2 model was loaded on first run and cached locally. The 449-test suite was executed with `pytest` for the daemon and `vitest` for the extension.

B. LABELLED EVALUATION CORPUS

We constructed four disjoint corpora: (1) **Benign** (50 packages): widely used, high-download packages with no known vulnerabilities; (2) **Typosquatted** (20 packages): hand-crafted variants of popular packages within edit distance 2; (3) **Hallucinated** (15 packages): names generated by prompting GPT-4 for npm utility packages and verified absent from the registry; (4) **Malicious** (15 packages): packages from public malware repositories [3] and the known-incident blocklist.

C. DETECTION PERFORMANCE

Table 2 reports precision, recall, and F1 for the WARN and BLOCK verdict classes, treating ALLOW as the negative class.

All hallucinated and malicious packages were correctly classified with perfect precision and recall. The three false positives on the benign corpus involved low-download packages that triggered Sentinel's reputation threshold; these would be resolved by a per-machine trust-list entry in production.

D. SCAN LATENCY

Table 3 reports end-to-end per-package scan latency (shim call to verdict return) across 100 trials per configuration.

The cache-miss, tarball-download P99 of 1 950 ms satisfies the sub-two-second target from design principle P5.

TABLE 3. Per-Package Scan Latency (ms, $N = 100$ per scenario)

Scenario	P50	P95	P99
Cache hit (SQLite)	18	24	31
Cache miss, pillar analysis	810	1 420	1 780
Cache miss, tarball download	1 140	1 790	1 950
Offline cache hit (daemon down)	11	18	24

TABLE 4. Automated Test Suite Summary ($N = 449$)

Module	Tests	Coverage
`test_sentinel.py`	78	93%
`test_shield.py`	62	90%
`test_shield_filescan.py`	41	88%
`test_aggregator.py`	55	97%
`test_contextify.py`	38	89%
`test_router.py`	49	91%
`test_diff_analyzer.py`	33	87%
`test_performance.py`	22	—
`test_drift_monitor.py`	21	92%
VS Code extension (vitest)	50	87%
Total / Daemon avg.	**449**	**91%**

E. ABLATION STUDY

We evaluated 12 weighting configurations varying $w_C \in \{0.20, 0.25, 0.30\}$ and distributing the remainder between w_S and w_{Sh} (Fig. **??**). The 0.30/0.35/0.35 configuration maximized macro-F1 (0.97) while keeping the benign false-positive rate below 5%. Configurations with $w_C \geq 0.40$ showed markedly higher false-positive rates on projects with sparse existing dependencies.

F. CONCEPT-DRIFT MONITOR VALIDATION

We injected 100 packages with uniformly random pillar scores into the audit log to simulate a shifted scoring environment. The monitor correctly raised `drift: "alert"` (KL divergence > 0.30 on all three pillars) within one health-check cycle (30 s). Restoring the natural score distribution caused the monitor to return to `drift: "ok"` after two cycles (60 s), confirming bi-directional responsiveness.

G. TEST SUITE COVERAGE

The 449-test suite achieves 91% line coverage on the daemon and 87% on the extension (Table 4).

IX. DISCUSSION

A. ACHIEVEMENT OF DESIGN PRINCIPLES

All five design principles from Section IV were validated. Pre-execution interception (P1) was confirmed by observing that no npm file download occurs before a BLOCK verdict is returned. Fail-open semantics (P2) were validated by bringing the daemon offline and confirming that the shim consulted the offline cache and proceeded for known packages, issuing a logged bypass for unknown ones. Explainability (P3) was confirmed through per-pillar scores and specific flag codes in every WARN dialog and REST response. Privacy preservation (P4) is ensured architecturally: the daemon makes outbound calls only to the npm registry, OSV.dev,

and HuggingFace (one-time model download). Scan latency satisfies P5 as shown in Table 3.

B. FAIL-OPEN VS. SECURITY TRADE-OFF

The fail-open philosophy introduces a deliberate tension with maximum enforcement. Our resolution—logging every bypass event to the tamper-evident audit log—preserves visibility without blocking developer workflows. This trade-off is appropriate for a developer-workstation tool; a complementary hard-fail CI gate would provide defense-in-depth.

C. FALSE POSITIVES AND TRUST LISTS

The three false positives observed on the benign corpus all involved low-download packages below Sentinel's reputation threshold. In a real deployment, a developer would add such packages to the per-machine trust list after a brief manual review. The HMAC protection ensures that pre-loading the trust list with malicious packages requires access to the machine secret, not merely write access to the SQLite file.

D. OBSERVATIONS ON MULTI-PILLAR DESIGN

The combination of independent, complementary signals proved more robust than any single heuristic. Typosquat detection alone produced occasional false positives on legitimately similarly-named packages, but requiring corroborating signal from reputation or contextual fit before escalating to BLOCK reduced unnecessary friction. This observation is consistent with findings on multi-signal aggregation in the malware detection literature [19].

E. LIMITATIONS

1) *No field deployment.* Conclusions about long-term false-positive tolerance and developer adoption are based on benchmark evaluation and manual testing, not longitudinal production data.
2) *Sparse-project Contextify signal.* The 50% weight reduction heuristic for small projects is pragmatic but not empirically validated.
3) *Static analysis evasion.* Novel or heavily obfuscated payloads may evade the Shield regex and tree-sitter pattern set; the rule set requires continuous expansion.
4) *Drift-monitor thresholds.* The KL divergence thresholds (0.15/0.30) were chosen heuristically without validation against a longitudinal production history.
5) *Optional LLM verifier.* The Ollama-based second-pass verifier was not included in the automated test paths and its accuracy on ambiguous cases has not been independently benchmarked.

F. FUTURE WORK

Priority future directions include: (i) a longitudinal field study across multiple development teams; (ii) expansion of the Shield rule set from newly discovered malicious packages; (iii) integration of the LLM verifier into the default scan path for high-ambiguity cases; (iv) validation and adaptive tuning of drift-monitor KL thresholds; (v) a complementary hard-fail CI gate mode; and (vi) post-install runtime permission enforcement [20].

X. CONCLUSION

We presented CIDAS, a real-time, pre-install npm package screening system that addresses four distinct supply-chain threat classes—typosquatting, malicious install scripts, maintainer-account compromise, and LLM package hallucination—within a single, locally-run, fail-open security framework. CIDAS's four-pillar architecture (Contextify, Sentinel, Shield, Aggregator) combines semantic project-context awareness, registry reputation analysis, name-similarity detection, and AST-based static scanning into explainable, policy-aware ALLOW/WARN/BLOCK verdicts. Supporting subsystems including an HMAC-protected trust list, a two-tier cache, an append-only audit log, and a KL-divergence-based concept-drift monitor provide ongoing assurance and team-wide governance.

Evaluated across 449 automated tests and a labelled 100-package corpus, CIDAS achieves a macro-F1 of 0.97 with sub-two-second per-package scan latency on commodity hardware, without requiring any cloud infrastructure or changes to existing developer workflows. A key finding is that the combination of independent, complementary signals is substantially more robust than any single detection heuristic—consistent with the broader trend toward multi-layer, shift-left supply-chain defenses.

.

APPENDIX A REST API ENDPOINT REFERENCE

Table 5 lists all ten CIDAS REST API endpoints.

TABLE 5. CIDAS REST API Endpoints

Endpoint	Description	Auth
`GET /health`	Daemon status & drift level	None
`POST /scan`	Analyze a package	Bearer
`GET /trust`	List trust-list entries	Bearer
`POST /trust`	Add a trust-list entry	Bearer
`POST /trust/verify`	Verify HMAC of an entry	Bearer
`GET /cache`	Inspect the scan cache	Bearer
`POST /cache/invalidate`	Force re-scan on next request	Bearer
`GET /audit`	Query the audit log	Bearer
`POST /audit/override`	Record a manual override	Bearer
`GET /policy`	Return resolved policy file	Bearer

APPENDIX B POLICY SCHEMA REFERENCE

- `block_list` (array of strings): Package names always BLOCKED.
- `trust_list` (array of strings): Package names that bypass all pillar analysis.
- `min_weekly_downloads` (integer, default 0): Reject packages below this download threshold.
- `require_repository` (boolean, default false): Require a `repository` field in `package.json`.

- `warn_requires_confirmation` (boolean, default false): Force interactive confirmation for WARN verdicts; non-interactive contexts treated as BLOCK.

ACKNOWLEDGMENT

The authors thank Dr. Vijayalakshmi M N (RVCE, Department of AIML) for guidance and the RVCE Experiential Learning programme for the opportunity to develop CIDAS as an SDG 9-aligned infrastructure project.

REFERENCES

[1] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2021. doi: 10.14722/ndss.2021.23055.

[2] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. 28th USENIX Secur. Symp.*, Santa Clara, CA, USA, Aug. 2019, pp. 995–1010.

[3] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. Detection Intrusions Malware, Vulnerability Assessment (DIMVA)*, ser. LNCS, vol. 12223, 2020, pp. 23–43. doi: 10.1007/978-3-030-52683-2_2.

[4] D.-L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, "Typosquatting and combosquatting attacks on the Python ecosystem," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroS&PW)*, Genoa, Italy, Sep. 2020, pp. 509–514. doi: 10.1109/EuroSPW51379.2020.00074.

[5] R. E. Zapata, R. G. Kula, B. Chinthanet, T. Ishio, K. Matsumoto, and A. Ihara, "Towards smoother library migrations: A look at vulnerable dependency migrations at function level for npm JavaScript packages," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Madrid, Spain, Sep. 2018, pp. 559–563. doi: 10.1109/ICSME.2018.00067.

[6] K. Huang, B. Chen, S. Wu, J. Cao, L. Ma, and X. Peng, "Demystifying dependency bugs in deep learning stack," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, San Francisco, CA, USA, Dec. 2023, pp. 450–462. doi: 10.1145/3611643.3616325.

[7] R. E. Zapata, R. G. Kula, B. Chinthanet, T. Ishio, K. Matsumoto, and A. Ihara, "Towards smoother library migrations," in *Proc. IEEE ICSME*, 2018, pp. 559–563.

[8] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Softw. Eng. Practice (ICSE-SEIP)*, Pittsburgh, PA, USA, May 2022, pp. 331–340. doi: 10.1109/ICSE-SEIP55303.2022.9794068.

[9] D.-L. Vu *et al.*, "On the feasibility of detecting software supply chain attacks," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, San Diego, CA, USA, Nov. 2021. doi: 10.1109/MILCOM52596.2021.9652901.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Nat. Lang. Process. (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[11] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, "We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs," in *Proc. 34th USENIX Secur. Symp. (USENIX Security 25)*, Seattle, WA, USA, Aug. 2025, pp. 3687–3706.

[12] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec '23)*, Copenhagen, Denmark, Nov. 2023, pp. 79–90. doi: 10.1145/3605764.3623985.

[13] J. Latendresse *et al.*, "An empirical study of account takeovers in the npm supply chain," in *Proc. IEEE/ACM Int. Conf. Mining Softw. Repositories (MSR)*, Pittsburgh, PA, USA, May 2022.

[14] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. ACM Conf. Fairness, Accountability, Transparency (FAT*)*, Barcelona, Spain, Jan. 2020, pp. 469–481. doi: 10.1145/3351095.3372828.

[15] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. 2023 IEEE Symp. Secur. Privacy (S&P)*, San Francisco, CA, USA, May 2023, pp. 1509–1526. doi: 10.1109/SP46215.2023.10179304.

[16] D.-L. Vu, Z. Newman, and J. S. Meyers, "Bad snakes: Understanding and improving Python package index malware scanning," in *Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Australia, May 2023, pp. 499–511. doi: 10.1109/ICSE48619.2023.00052.

[17] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings ACL: EMNLP 2020*, Online, Nov. 2020, pp. 1536–1547. doi: 10.18653/v1/2020.findings-emnlp.139.

[18] J. Cappos, J. Samuel, S. Baker, and J. H. Hartman, "A look in the mirror: Attacks on package managers," in *Proc. 15th ACM Conf. Comput. Commun. Secur. (CCS '08)*, Alexandria, VA, USA, Oct. 2008, pp. 565–574. doi: 10.1145/1455770.1455841.

[19] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in *Proc. 44th IEEE/ACM Int. Conf. Softw. Eng. (ICSE '22)*, Pittsburgh, PA, USA, May 2022, pp. 1681–1692. doi: 10.1145/3510003.3510104.

[20] G. Ferreira, L. Jia, J. Sunshine, and C. Kästner, "Containing malicious package updates in npm with a lightweight permission system," in *Proc. 43rd IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Madrid, Spain, May 2021, pp. 1334–1346. doi: 10.1109/ICSE43902.2021.00121.

[21] F. N. Froh, M. F. Gobbi, and J. Kinder, "Differential static analysis for detecting malicious updates to open source packages," in *Proc. ACM Workshop Softw. Supply Chain Offensive Res. Ecosystem Defenses (SCORED '23)*, Copenhagen, Denmark, Nov. 2023, pp. 41–49. doi: 10.1145/3605770.3625211.